Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport
la
Lucrarea de laborator Nr. 2

Disciplina: Tehnici de programare

Tema: ” Lucrul în linia de comandă Linux ”

A efectuat : st. gr.SI – 265 Spătaru Marius

A verificat : asistent universitar Cristian Gonceari

Chișinău – 2026

REGULI DE PREDARE A LUCRĂRII

Citește această pagină înainte de a începe. Nerespectarea regulilor de mai jos înseamnă că lucrarea nu poate fi identificată și nu este primită.

1. Cum se numește fișierul pe care îl trimiți

Denumirea fișierului trebuie să respecte exact acest tipar:

`L2_TP_Nume_Prenume_Grupa.docx`

Exemplu, pentru studentul Popescu Ion din grupa SI-266:

`L2_TP_Popescu_Ion_SI-266.docx`

Partea	Ce scrii	Exemplu
L2	Litera L și numărul laboratorului. La această lucrare este L2.	L2
TP	Abrevierea disciplinei, cu majuscule. Se scrie identic.	TP
Nume	Numele de familie, cu majusculă la început, fără diacritice.	Popescu
Prenume	Prenumele, cu majusculă la început, fără diacritice.	Ion
Grupa	Grupa, cu majuscule și cu cratimă.	SI-266
.docx	Extensia. Numai Word. Nu se acceptă PDF.	.docx

2. Cum trimiți lucrarea

	Regula
Adresa de email	cristian.gonceari@isa.utm.md
Subiectul emailului	L2_TP_Popescu_Ion_SI-266
	Adică exact denumirea fișierului, dar fără „.docx”. Nimic altceva în subiect.
Atașament	Un singur fișier .docx. Fără arhive, fără poze separate.
Corpul emailului	Nu este nevoie să scrii nimic.

De ce contează subiectul: cu acest subiect, toate lucrările unui laborator pot fi găsite și sortate instantaneu. Un subiect scris liber înseamnă că lucrarea ta se poate pierde printre celelalte.

3. Cum completezi acest document

1. Completează pagina de titlu. Tema lucrării este stabilită de profesor și nu se modifică. Tu înlocuiești doar textul evidențiat cu galben — grupa, numele și prenumele tău — apoi ștergi evidențierea galbenă.

2. Parcurge sarcina lucrării de la pagina următoare, pe platforma pwn.college.

3. Răspunde la toate cele 20 de întrebări, în chenarele pregătite sub fiecare întrebare. Scrie cu cuvintele tale.

4. Completează secțiunea de concluzii.

5. Salvează fișierul cu denumirea de la punctul 1 și trimite-l pe email.

SARCINA LUCRĂRII

Parcurge pe platforma pwn.college dojo-ul Linux Luminarium și rezolvă integral modulele obligatorii: Hello Hackers, Pondering Paths, Comprehending Commands, Digesting Documentation, File Globbing și Practicing Piping. Rezolvă apoi cel puțin patru dintre modulele recomandate: Shell Variables, Data Manipulation, Processes and Jobs, Untangling Users și Perceiving Permissions. Pe parcursul lucrului notează-ți comenzile exacte pe care le-ai rulat, mesajele de eroare primite și modul în care ai depășit fiecare blocaj, fiindcă întrebările din acest raport se referă la experiența ta directă, nu la definiții. Răspunde apoi, în acest document, la toate cele 20 de întrebări, cu propriile cuvinte și cu exemple concrete din sesiunile tale de lucru. Răspunsurile copiate din manuale sau de pe internet, care nu au legătură cu ceea ce ai rulat efectiv, nu se punctează.

Platforma	Cursul de parcurs	Adresa
pwn.college	Linux Luminarium	https://pwn.college/linux-luminarium/

Modulele pe care le parcurgi

Nr.	Modul	Challenge-uri	De rezolvat	Nivel
1	Hello Hackers	3	3	OBLIGATORIU
2	Pondering Paths	8	8	OBLIGATORIU
3	Comprehending Commands	15	15	OBLIGATORIU
4	Digesting Documentation	7	7	OBLIGATORIU
5	File Globbing	10	10	OBLIGATORIU
6	Practicing Piping	15	15	OBLIGATORIU
7	Shell Variables	8	8	RECOMANDAT
8	Data Manipulation	6	6	RECOMANDAT
9	Processes and Jobs	10	10	RECOMANDAT
10	Untangling Users	4	4	RECOMANDAT
11	Perceiving Permissions	8	8	RECOMANDAT
12	Chaining Commands	12	12	BONUS
13	Terminal Multiplexing	6	6	BONUS
14	Pondering PATH	5	5	BONUS
15	Silly Shenanigans	6	6	BONUS
16	Daring Destruction	5	5	BONUS

Modulele marcate OBLIGATORIU conțin materia pe care se bazează întrebările. Cele marcate RECOMANDAT și BONUS te ajută la întrebările mai dificile.

ÎNTREBĂRI

Răspunde la toate cele 20 întrebări, în chenarele de mai jos. Fiecare răspuns corect valorează 0,5 puncte.

1. La exercițiul Intro to Arguments ai rulat comanda hello mai întâi fără argument, apoi cu argumentul hackers. Explică ce face shell-ul concret cu textul pe care îl scrii la prompt înainte de a porni programul și de ce programul a reacționat diferit în cele două cazuri.

(modulul Hello Hackers)

Când scriu o comandă la prompt, shell-ul sparge textul în cuvinte separate prin spații înainte să lanseze programul. Primul cuvânt este numele executabilului, iar restul devin argumente pe care le primește acel program. La prima rulare am scris doar `hello`, așa că programul nu a primit niciun argument și mi-a afișat mesajul că așteaptă un parametru. La a doua rulare am scris `hello hackers`, iar shell-ul i-a transmis textul `hackers` ca argument, ceea ce a făcut programul să meargă mai departe și să-mi afișeze flag-ul.

2. La exercițiul Command History flag-ul a fost introdus în istoricul shell-ului tău. Relatează concret cum ai ajuns la el: ce ai apăsat sau ce comandă ai folosit, ce a apărut pe ecran și unde se păstrează acest istoric.

(modulul Hello Hackers)

Am folosit comanda `history` în terminal și mi-a afișat pe ecran lista numerotată cu ultimele comenzi rulate. Am derulat prin listă și am găsit comanda anterioară care conținea flag-ul. În Linux, acest istoric este salvat în fișierul ascuns `.bash_history` din directorul meu principal (`~/.bash_history`).

3. Aflându-te chiar în directorul /challenge, comanda scrisă doar cu numele programului eșuează, iar varianta cu ./ în față funcționează. Explică ce caută shell-ul în fiecare dintre cele două cazuri.

(modulul Pondering Paths)

Când scriu doar numele comenzii fără cale, shell-ul o caută doar în directoarele de sistem salvate în variabila de mediu `$PATH` (cum ar fi `/usr/bin`). Pentru că directorul curent `/challenge` nu se află în `$PATH`, sistemul zice că nu găsește comanda. Când pun `./` în față, îi spun explicit shell-ului să caute și să execute fișierul chiar din directorul curent în care mă aflu.

4. La exercițiile implicit relative paths, from / și implicit relative path a trebuit să rulezi același program o dată din directorul / și o dată din /challenge, scriind de fiecare dată altceva. Explică de la ce

punct pornește sistemul când primește o cale relativă și cum se schimbă calea pe care trebuie să o scrii atunci când te muți cu cd.

(modulul Pondering Paths)

O cale relativă pornește mereu din directorul de lucru curent, adică din locul în care mă aflu în acel moment. Când eram în `/`, am scris `challenge/run` ca să ajung la fișier. După ce m-am mutat cu `cd /challenge`, punctul de plecare s-a schimbat, așa că nu mai trebuia să scriu `challenge/`, ci am folosit direct `./run`.

5. La exercițiul home sweet home aveai voie să dai un argument de cel mult trei caractere. Scrie exact ce ai introdus ca argument și explică prin ce mecanism acele câteva caractere ajung să indice un fișier din directorul tău personal.

(modulul Pondering Paths)

Am scris ca argument `~/a`. Tasta `~` este prescurtarea pe care shell-ul o extinde automat în calea completă către directorul meu de utilizator (`/home/hacker`). Astfel, deși eu am scris doar 3 caractere la consolă, programul a primit calea întreagă `/home/hacker/a` și a trecut de limita de lungime.

6. Numele comenzii cat vine de la concatenare. Pe baza exercițiilor pe care le-ai rulat în Comprehending Commands, explică ce se întâmplă când îi dai mai multe fișiere ca argumente și cum se vede acest lucru în ieșirea pe care ai obținut-o.

(modulul Comprehending Commands)

Comanda `cat` citește și lipește conținutul fișierelor primite ca argumente. Când i-am dat două fișiere, de exemplu `cat fisier1 fisier2`, le-a citit pe rând și a afișat conținutul lor unul după altul pe ecran, fără nicio pauză sau linie separatoare între ele.

7. La exercițiul An Epic Filesystem Quest ai urmărit indicii dintr-un director în altul până la flag. Relatează concret traseul tău: ce pereche de comenzi ai repetat la fiecare pas și cum ai decis încotro mergi mai departe.

(modulul Comprehending Commands)

La fiecare pas am folosit `ls -a` ca să văd ce fișiere sunt în director (inclusiv cele ascunse) și `cat` pe fișierul de indiciu găsit. Textul din indiciu îmi spunea exact în ce director să merg mai departe. Foloseam `cd` către acel director și repetam aceeași combinație (`ls -a` și `cat`) până când am ajuns la fișierul final care conținea flag-ul.

8. La exercițiul linking files ai creat o legătură simbolică cu ln -s. Explică ce conține de fapt fișierul-legătură și ce se întâmplă în momentul în care rulezi cat pe el.

(modulul Comprehending Commands)

Fișierul legătură simbolică (*symlink*) nu conține datele reale, ci doar o adresă de text care arată către fișierul țintă. Când rulez `cat` pe el, sistemul vede că este o legătură, urmărește automat calea salvată în el și îmi afișează conținutul fișierului spre care arată.

9. Compară cele trei surse de documentație folosite în Digesting Documentation: man, opțiunea --help și comanda help. Spune pentru ce fel de comandă se potrivește fiecare și de ce pentru comanda-builtin din ultimul exercițiu nu ai putut folosi man.

(modulul Digesting Documentation)

`man` se folosește pentru utilitare de sistem separate (cum e `ls` sau `cat`). Opțiunea `--help` se dă scurt la comenzi pentru a vedea rapid opțiunile lor. Comanda `help` este specială pentru comenzile interne ale shell-ului (cum e `cd`). Pentru comanda builtin nu a mers `man` pentru că nu este un fișier executabil separat pe disc care să aibă o pagină de manual proprie.

10. La exercițiul Searching For Manuals pagina de manual avea un nume generat aleator. Spune ce comandă ai folosit ca să o găsești, după ce cuvânt ai căutat și cum arăta rezultatul afișat de acea căutare.

(modulul Digesting Documentation)

Am folosit comanda `man -k flag` ca să caut prin descrierile tuturor paginilor de manual. Căutarea mi-a returnat o listă de linii unde apărea cuvântul `flag`, iar una dintre linii conținea numele generat aleator al paginii și o scurtă descriere. Apoi am deschis-o rulând `man` urmat de acel nume.

11. Explică cine face expandarea unui tipar precum /challenge/f*: shell-ul sau programul apelat. Argumentează pe baza a ceea ce ai observat efectiv, inclusiv ce s-a întâmplat când niciun fișier nu se potrivea tiparului.

(modulul File Globbing)

Expandarea tiparelor cu * este făcută de **shell**, nu de programul apelat. Shell-ul înlocuiește `f*` cu lista tuturor fișierelor care încep cu `f` și le trimite programului ca argumente. Am văzut că dacă nu există niciun fișier care să se potrivească, shell-ul lasă textul exacta așa cum l-am scris (`/challenge/f*`), iar programul dă eroare pentru că nu găsește un fișier cu stelucă în nume.

12. Compară folosirea unui glob cu folosirea completării prin tasta Tab pentru a ajunge la același fișier. Spune în ce situație concretă din modul ai preferat una și în ce situație cealaltă, și de ce completarea prin Tab este considerată mai sigură.

(modulul File Globbing)

Punctul forte la * (globbing) este că pot potrivi mai multe fișiere deodată dintr-o singură comandă. Tasta `Tab` o folosesc când vreau să scriu rapid o cale lungă fără să fac greșeli de tipar. `Tab` este mai sigură pentru că îmi arată pe ecran că fișierul chiar există înainte să apăs Enter, prevenind executarea comenzii pe fișiere greșite.

13. Descrie ce se întâmplă concret cu textul dintre două programe legate prin pipe, pe exemplul pe care l-ai rulat tu la exercițiul Grepping live output. Precizează de unde citește al doilea program și dacă textul mai trece prin vreun fișier de pe disc.

(modulul Practicing Piping)

La o comandă de tipul `/challenge/run | grep pwn`, textul scos de primul program trece direct în intrarea celui de-al doilea prin intermediul memoriei RAM gestionate de sistem. Al doilea program (`grep`) citește textul direct din fluxul de intrare standard (`stdin`). Datele nu sunt salvate și nu trec prin niciun fișier de pe hard disk.

14. Ai redirecționat ieșirea programului /challenge/run într-un fișier folosind operatorul > și totuși pe ecran au continuat să apară mesaje. Explică de ce au apărut acele mesaje și ce ai scris ca să le capturezi și pe ele.

(modulul Practicing Piping)

Mesajele au apărut pe ecran pentru că erau mesaje de eroare trimise pe fluxul `stderr` (descriptorul 2), în timp ce simbolul `>` redirecționează doar ieșirea normală `stdout` (descriptorul 1). Ca să le capturez pe toate în fișier, am adăugat `2>&1` la finalul comenzii sau am folosit `&>`.

15. Modulul îți cere mai întâi să salvezi ieșirea într-un fișier și apoi să o filtrezi cu grep, iar la exercițiul următor să filtrezi direct prin pipe. Compară cele două metode și spune în ce situație reală ai alege salvarea în fișier și în ce situație pipe-ul direct.

(modulul Practicing Piping)

Salvarea într-un fișier este utilă când procesul durează mult și vreau să păstrez rezultatul pe disc ca să-l pot analiza mai târziu fără să mai rulez comanda o dată. Pipe-ul direct este mai bun când am nevoie rapid de un rezultat (de exemplu când caut un text scurt cu `grep`) și nu mă interesează să salvez datele intermediare

16. La exercițiul Exporting Variables ai setat o variabilă, iar programul lansat de tine nu o vedea până când nu ai folosit export. Explică diferența dintre o variabilă de shell obișnuită și una exportată, din perspectiva procesului-copil.

(modulul Shell Variables)

O variabilă simplă de shell rămâne doar în sesiunea curentă și nu este văzută de programele pe care le pornesc din ea. Când folosesc `export`, variabila este adăugată în mediul de lucru global al shell-ului, așa că orice program lansat ulterior din acel terminal o va moșteni și o va putea citi.

17. La exercițiile din Data Manipulation flag-ul îți venea stricat, cu caractere în plus, cu linii noi intercalate sau cu majusculele inversate. Alege unul dintre aceste exerciții, scrie comanda exactă pe care ai folosit-o pentru a repara ieșirea și spune cum arăta textul înainte și după.

(modulul Data Manipulation)

La exercițiul unde textul venea cu caractere nedorite sau linii goale, am folosit comanda `tr` legată prin pipe: `/challenge/run | tr -d '\n'`. Înainte de această comandă, ieșirea era spartă pe mai multe rânduri și nu putea fi citită ca un flag valid. După ce am curățat caracterele de linie nouă, textul s-a unificat într-un singur rând clar cu formatul `pwn.college{...}`

18. Ai apăsat Ctrl-Z peste un program care rula. Explică în ce stare ajunge procesul imediat după această apăsare și de ce el nu își continuă lucrul până când nu dai comanda bg sau fg.

(modulul Processes and Jobs)

După ce apăs `Ctrl-Z`, procesul este pus în pauză (*Stopped*) și trimis în fundal. El nu își continuă munca pentru că sistemul îi oprește alocarea de timp de procesor până când nu primește comanda de

reluare. Ca să îl fac să meargă mai departe, trebuie să dau comanda `bg` (pentru fundal) sau `fg` (pentru prim-plan).

19. La exercițiul The SUID Bit ai adăugat bitul SUID unui program și abia apoi el a putut citi flag-ul. Explică cu ce identitate rulează un program obișnuit față de unul cu bit SUID și de ce asta schimbă rezultatul verificării drepturilor la citirea fișierului.

(modulul Perceiving Permissions)

Un program obișnuit rulează cu drepturile utilizatorului care l-a pornit. Un program cu bitul SUID setat (`chmod u+s`) rulează cu drepturile proprietarului fișierului (care în acest caz era `root`). Când am adăugat bitul SUID, programul a putut citi fișierul flag pentru că rulat automat cu permisiunile de `root`.

20. În modulul Untangling Users ai schimbat identitatea o dată cu su și o dată cu sudo. Explică ce parolă ți s-a cerut în fiecare dintre cele două cazuri și pe ce se bazează sistemul când decide dacă îți permite sau nu operația.

(modulul Untangling Users)

La comanda `su` sistemul mi-a cerut parola contului la care încercam să mă conectez. La comanda `sudo` mi-a cerut parola contului meu curent. La `su` accesul depinde doar de introducerea parolei corecte a acelui utilizator, iar la `sudo` sistemul verifică mai întâi dacă utilizatorul meu are dreptul de a folosi comenzi de administrator în fișierul de configurare `/etc/sudoers`.

CONCLUZII

Scrie 10 – 15 rânduri: ce ai învățat concret, ce ți s-a părut cel mai dificil, unde te-ai blocat și cum ai ieșit din blocaj. Evită formulările generale de tipul „lucrarea a fost utilă”.

Pe parcursul celor 10 module din Linux Luminarium de pe pwn.college am învățat modul de lucru practic în linia de comandă Bash. Am înțeles cum procesează shell-ul argumentele, cum se navighează corect folosind căi absolute și relative și cum se pot lega comenzile între ele prin pipe-uri și redirecționări. Cel mai mult m-am blocat la modulul de piping, mai ales când trebuia să redirecționez separat erorile (`stderr`) de ieșirea obișnuită (`stdout`). La început mi-a fost mai greu și cu căile relative, unde uitam să verific în ce director mă aflu înainte de a rula comanda. Am depășit aceste dificultăți citind atent erorile afișate în terminal, verificând locația curentă cu `pwd` și folosind opțiunile de ajutor (`--help` sau `man`). Această experiență m-a ajutat să nu mai scriu comenzi „după ureche”, ci să înțeleg ce se întâmplă în spatele lor în sistemul de operare.

BIBLIOGRAFIE

1. pwn.college (Arizona State University), Linux Luminarium, [Online]. Disponibil: https://pwn.college/linux-luminarium/

2. pwn.college, linux-luminarium (codul sursă și descrierile modulelor și ale challenge-urilor), GitHub, [Online]. Disponibil: https://github.com/pwncollege/linux-luminarium

3. Free Software Foundation, GNU Bash Reference Manual, [Online]. Disponibil: https://www.gnu.org/software/bash/manual/bash.html

4. M. Kerrisk, The Linux man-pages project (paginile de manual ale comenzilor Linux), [Online]. Disponibil: https://man7.org/linux/man-pages/

5. W. Shotts, The Linux Command Line: A Complete Introduction, No Starch Press, [Online]. Disponibil: https://linuxcommand.org/tlcl.php

6. The Linux Foundation, Filesystem Hierarchy Standard (FHS), versiunea 3.0, [Online]. Disponibil: https://refspecs.linuxfoundation.org/FHS_3.0/fhs/index.html

BAREM DE NOTARE

Nota se calculează numai din răspunsurile la cele 20 de întrebări.
20 întrebări × 0,5 puncte = 10 puncte = nota 10

Cum se punctează un răspuns

Punctaj	Când se acordă
0,5 puncte	Răspuns corect și complet: explică mecanismul, nu doar enunță rezultatul.
0,25 puncte	Răspuns parțial corect: ideea principală este bună, dar explicația este incompletă sau conține o imprecizie.
0 puncte	Răspuns greșit, lipsă, sau copiat cuvânt cu cuvânt de la alt student ori dintr-o sursă de pe internet.

Cum se transformă punctajul în notă

Răspunsuri corecte	Punctaj	Nota
20 răspunsuri corecte	10 puncte	10
18 – 19 răspunsuri corecte	9 – 9,5 puncte	9
16 – 17 răspunsuri corecte	8 – 8,5 puncte	8
14 – 15 răspunsuri corecte	7 – 7,5 puncte	7
12 – 13 răspunsuri corecte	6 – 6,5 puncte	6
10 – 11 răspunsuri corecte	5 – 5,5 puncte	5
sub 10 răspunsuri corecte	sub 5 puncte	nepromovat — lucrarea se reface